LISN
LABORATOIRE INTERDISCIPLINAIRE DES SCIENCES DU NUMÉRIQUE

cnrs université PARIS-SACLAY

# LLM agentique et vibe coding

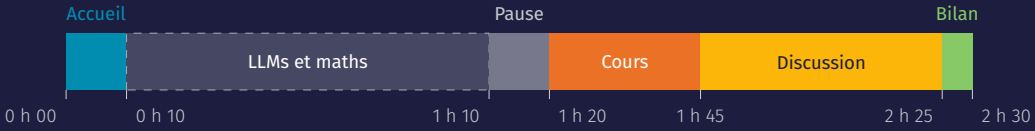
## EcAuTAL 2026, école d'automne en TAL du département STL

Luc Pommeret

LISN, Orsay, octobre 2026

CNRS, LISN, Université Paris-Saclay

# Programme de la séance



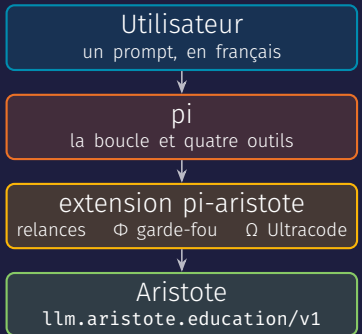

| Début | Durée | Séquence |
|---|---|---|
| 0 h 00 | 10 min | Accueil, installation de pi-aristote |
| 0 h 10 | 60 min | Les LLMs et les problèmes de maths (*Claude's Cycles*, Erdős, RMS) |
| 1 h 10 | 10 min | Pause |
| 1 h 20 | 25 min | Cours : LLM agentique (MCP, sous-agents), vibe coding |
| 1 h 45 | 40 min | Discussion |
| 2 h 25 | 5 min | Bilan, questions |

# Installation de pi-aristote

# pi-aristote (l'agent pi branché sur Aristote)

- pi : agent de code libre (licence MIT), qui lit, écrit et exécute dans le dossier courant
- Aristote : passerelle de LLM ouverts (Qwen, Gemma, Llama) portée par CentraleSupélec, une clé par compte universitaire
- pi-aristote : l'extension qui branche pi sur Aristote (modèles à jour, relances automatiques, garde-fou, sous-agents)

Outil du TP → à installer maintenant



# Clé API et installeur

## α Clé API

1. Ouvrir `aristote.education/llm-access/`
2. Se connecter avec son compte universitaire
3. Copier la clé (elle commence par `sk-`)

## L'installeur

- Node.js privé si besoin, puis pi, l'extension, le thème
- Vérifie la clé et la range dans `~/.pi/agent`
- Sans `sudo`, et ne remplace jamais une configuration existante sans demander

## β Installer

```
# macOS, Linux : dans un terminal
$ curl -fsSL https://vibe-coding.pages.lisn.upsaclay.fr/pi-aristote/install.sh | sh
# Windows : dans PowerShell, une fois WSL installé (wsl --install, puis redémarrer)
PS> wsl bash -lc "curl -fsSL https://vibe-coding.pages.lisn.upsaclay.fr/pi-aristote/install.sh | sh"
```

# Premier lancement

Terminal

```
$ mkdir -p ~/ecautal && cd ~/ecautal     # un dossier dédié
$ pi
› /aristote                              # clé, modèles, contexte
› /aristote key sk-...                   # si la clé manque ou a changé
› Que contient ce dossier ? Réponds en une phrase.
```

*Changer de modèle* :

- `Ctrl+L` ou **`/model`** : la liste des modèles
- `Ctrl+S` dans la liste : en faire le modèle par défaut
- `Échap` interrompt l'agent à tout moment

| Modèle (Aristote) | | Contexte |
|---|---|---|
| `qwen-3.8-27b` | par défaut | 262 144 |
| `qwen-3.6-35b-instruct` | | 262 144 |
| `gemma-4-31b` | | 262 144 |
| `llama-3.1-8b` | | 131 072 |

(liste relue à chaque lancement, modèles en panne masqués)

# Supports en ligne



- Site et installeur
  `vibe-coding.pages.lisn.upsaclay.fr/pi-aristote`
- Code de l'extension (en accès libre)
  `gitlab.lisn.upsaclay.fr/vibe-coding/pi-aristote`

*« Pour résoudre une question ou terminer une controverse, les adversaires n'auront qu'à prendre la plume, en s'adjoignant au besoin un ami comme arbitre, et à dire :* « Calculons ! » »

*— Couturat, La Logique de Leibniz (1901), paraphrasant Leibniz*

# I. Panorama des outils LLM

# Le *calculus ratiocinator* de Leibniz

Leibniz distingue deux composantes d'un projet de mécanisation du raisonnement :

- *Characteristica universalis* : un langage formel universel encodant les concepts.
- *Calculus ratiocinator* : un calcul opérant sur ce langage, réduisant tout raisonnement à de la manipulation symbolique.

*Héritiers naturels* : la logique formelle (Frege, Russell), les prouveurs automatiques, l'IA symbolique classique.

**Paradoxe** : les LLMs ne procèdent **pas** par réécriture symbolique, mais par statistique sur des représentations latentes. Et pourtant, ils semblent "calculer". Réalisent-ils le *calculus ratiocinator* par des moyens que Leibniz n'avait pas envisagés ?

# LLM et Transformers

*Transformer* (Vaswani et al., 2017) : architecture neuronale basée sur le mécanisme d'*attention*.

*Principe* : prédire le token suivant $x_{t+1}$ étant donné le contexte $x_1, \ldots, x_t$. Prédiction d'une distribution de probabilité sur le vocabulaire, puis tirage.

*LLM* (Large Language Model) :

- Transformer à grande échelle (milliards de paramètres)
- Pré-entraîné sur des corpus massifs (web, livres, code)
- Capacités émergentes : raisonnement, few-shot learning, génération de code

*Exemples* :

$$\text{GPT}_n \ (n \in \{3, 3.5, 4, 4.5, 5\}), \quad \text{Claude}_n \ (n \in \{3, 3.5, 3.7, 4, 4.5\}), \quad \text{Gemini}_n \ (n = 1 + \tfrac{k}{2})$$

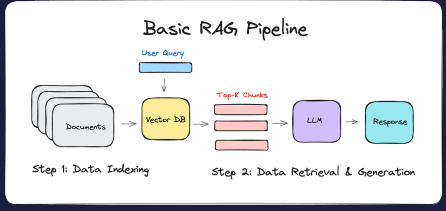
# RAG (Retrieval-Augmented Generation)

*Définition* : architecture combinant un LLM et une base de connaissances externe.

*Processus* :

1. *Indexation* : le corpus est projeté dans un espace vectoriel
2. *Récupération* : recherche des documents pertinents par similarité
3. *Génération* : le modèle répond en s'appuyant sur le contexte récupéré

*Objectif* : réduire les hallucinations, étendre le contexte.



# Agents, MCP et Vibe Coding

*Agents* (Wang et al., 2024) : le LLM comme contrôleur cognitif autonome.

- Planification (Chain of Thought, Reasoning)
- Mémoire : court terme (attention) + long terme (RAG)
- Outils via *MCP* (Model Context Protocol, Anthropic 2024)

*Vibe Coding* (Karpathy, fév. 2025) :

- Génération de code par dialogue avec un LLM
- Risque d'hallucination → besoin de vérification
- Brute force : le *Ralph Loop* (Huntley, 2025)




*Synthèse* : le LLM génère des esquisses, l'assistant de preuve (Lean, Rocq) vérifie.

# II. *Claude's Cycles* (Knuth, 2026)

# *Claude's Cycles* : le problème

*Contexte* : Knuth travaille sur les cycles hamiltoniens dirigés pour un futur volume de *The Art of Computer Programming*.

*Le problème* : soit le digraphe à $m^3$ sommets $ijk$ ($0 \leq i, j, k < m$), avec trois arcs depuis chaque sommet (incrémentation de $i$, $j$ ou $k$ mod $m$). Décomposer l'ensemble des arcs en **trois cycles hamiltoniens dirigés**, pour tout $m > 2$.

*État initial* :

- Knuth avait résolu le cas $m = 3$.
- Filip Stappers avait trouvé empiriquement des solutions pour $4 \leq m \leq 16$.
- Conjecture : la décomposition existe pour tout $m > 2$.

# *Claude's Cycles* : 31 explorations

Stappers soumet le problème à **Claude Opus 4.6**. En $\sim$**1 heure**, 31 explorations :

| | |
|---|---|
| **Reformulation** | Claude identifie un graphe de Cayley. |
| **Expl. 1–2** | $\sigma$ cyclique avec $g$ linéaire ou quadratique : échec. DFS brute-force : $6^{27}$, trop lent. |
| **Expl. 3** | **Premier éclair** : redécouvre le *serpentin 2D*. |
| **Expl. 4** | Généralisation 3D via le *code de Gray m-aire* (Knuth, *TAOCP* 4A, p. 299) : un cycle, mais digraphe résiduel trop rigide. |
| **Expl. 15** | **Le déclic** : décomposition en *fibres* via $\pi(i,j,k) = i+j+k \bmod m$. Le problème 3D se ramène à un empilement de problèmes 2D. |
| **Expl. 25** | Auto-diagnostic : "*SA can find solutions but cannot give a general construction. Need pure math.*" |
| **Expl. 31** | **Succès** : dépendance en une seule coordonnée par fibre. Valide pour tout $m$ **impair**, testé jusqu'à $m = 101$. |

*Rôle humain* : Stappers relance, documente, guide.
"*After EVERY exploreXX.py run, IMMEDIATELY update plan.md before doing anything else.*"

# *Claude's Cycles* : résultats et suites

*Formalisation* :

- Knuth rédige une preuve rigoureuse de la construction de Claude.
- **Kim Morrison** (communauté Lean) formalise cette preuve en Lean en 24h.

*Le cas pair* ($m$ pair, $m \geq 8$) :

- Ho Boon Suan obtient une solution via **GPT-5.3 Codex**.
- La preuve de correction est produite par **GPT-5.4 Pro** : 14 pages, sans édition humaine.

*Multi-agent* : Exocija trouve une construction plus simple en alternant **GPT 5.4** et **Claude 4.6 Sonnet**.

Knuth : "*Hats off to Claude! [...] We are living in very interesting times indeed.*"

# III. IA et formalisation/découverte mathématique

# Critères de la découverte (Tao, 2025)

Une contribution de l'IA est une découverte si :

1. La preuve est *nouvelle* (non présente dans la littérature mathématique)
2. Elle est *générée de manière autonome* (sans intervention humaine dans la fabrication)
3. Elle est *formalisée* et vérifiée (ici en Lean)

# Dean et Naibo (2025)

*"Artificial Intelligence and Inherent Mathematical Difficulty"*
Philosophia Mathematica, juillet 2025.

*Thèse (N0)* : "Advances in computing theory or technology – inclusive of those currently understood to exemplify artificial intelligence – will not radically alter the following aspect of our current understanding of mathematics and its practice : Settling the status of open questions is an inherently difficult problem."

*Thèse (N1)* : "The application of artificial intelligence will not lead to a substantial reduction in the difficulty of one of the central goals of mathematical practice."

Arguments :

- La difficulté est structurelle (indécidabilité, NP-complétude)
- L'IA excelle dans la recherche ($\mathbf{\Sigma}^0_1$), pas dans la conceptualisation

# Complexité de découverte vs complexité de vérification

*Complexité de vérification* :

- Vérifier qu'une suite de symboles est une preuve valide
- Souvent polynomiale (P)

*Complexité de découverte* (Detlefsen, 1990) :

- Trouver le chemin vers la preuve
- Souvent exponentielle ou pire

# La hiérarchie arithmétique reste la contrainte de l'IA (Dean et Naibo, 2025)

$\Sigma_1^0 : \exists x\, \phi(x)$

- Recherche existentielle
- Accessible à l'IA (recherche de témoin)

$\Pi_1^0 : \forall x\, \phi(x)$

- Propriété universelle
- Réfutation = trouver un contre-exemple ($\Sigma_1^0$)

$\Pi_2^0 : \forall x\, \exists y\, \phi(x, y)$

- Alternance de quantificateurs
- Nécessite un argument générique

# La *Septimana Mirabilis* (4–12 janvier 2026)

**4 janvier** Kevin Barreto et “AcerFur” soumettent #728 à GPT-5.2 Pro.

**4–6 janvier** Aristotle traduit en Lean. Correction autonome.

**8 janvier** Terence Tao valide. Statut : **PROVED (LEAN)**.

**10 janvier** Résolution autonome du problème **#205** par Barreto avec GPT-5.2 Thinking et Aristotle.

**11–12 janv.** Réfutation du #397 et résolution du #729.

# Le coup décisif : le Problème #205

*Conjecture originale (Erdős)* : tout $n$ suffisamment grand peut-il s'écrire $2^k + m$ avec $\Omega(m) < \log\log m$ ?

*Résultat IA (10 jan 2026)* : Réfutation quantitative. Il existe une infinité de $n$ tels que pour tout $k$ avec $2^k < n$, $\Omega(n - 2^k) \gg \sqrt{\frac{\log n}{\log\log n}}$.

*Méthode* :

- Théorème des restes chinois (CRT) pour construire $n$.
- Utilisation du Théorème des Nombres Premiers (PNT).
- Argument par cas universel.

`https://www.erdosproblems.com/205`

# Analyse logique du #205

L'énoncé formel exact, validé en Lean (`infinitely_many_counterexamples`) :

$$\exists c > 0, \forall N, \exists n > N, \forall k, (2^k \leq n \to \Omega(n - 2^k) \geq c \cdot \text{pntRate}(n))$$

*Classification* : $\Pi_3^0$ (ou $\Sigma_4^0$ avec le paramètre réel $c$).

- L'alternance interne $\exists n \forall k$ est irréductible.
- La preuve ne procède **pas** par force brute ou énumération, mais par construction analytique.
- Réfute la thèse selon laquelle seuls des problèmes $\Sigma_1^0$ (force brute déguisée) seraient accessibles aux LLMs (et donc aux IAs).

# IV. Un cas limite : de la traduction à la compilation

# Dépasser la notion de compilation ?

*Projet expérimental* : Traducteur de pseudo-code vers Python

- Finetuning d'un petit LLM (Qwen3.5 0.8B).
- Mode **déterministe** : exécution sur CPU avec température 0.
- Tâche : convertir du pseudo-code en anglais naturel vers du code Python exécutable.

*Question* : est-ce un compilateur ?

- La « formalisation » est régie par des probabilités linguistiques plutôt que des règles syntaxiques strictes.
- Cas limite : reconsidérer l'opposition entre compilation déterministe et traduction automatique.

`https://gitlab.lisn.upsaclay.fr/pommeret/pseudo-code`

# Espace latent vs formalisation sémantique

*Correspondance de Curry-Howard* : dans un assistant de preuve (Lean, Rocq), un programme **est** une preuve. La formalisation est *sémantique* — chaque terme a une signification logique vérifiable.

*Le LLM à température 0* :

- Le pseudo-code est projeté dans un **espace latent** (représentation vectorielle continue).
- La « compréhension » est *géométrique*, pas logique : proximité dans l'espace $\neq$ équivalence formelle.
- Pourtant : le code produit est correct et déterministe.

*Tension* : le LLM produit une formalisation *de facto* (du code exécutable) sans formalisation *de jure* (pas de correspondance preuves-programmes). Une sémantique latente plutôt que logique.

# Démonstration

lancer la commande ”pc exemple.pc”

Le compilateur interprète et traduit l’intention structurelle.

# Conclusion

# Méta-induction pessimiste et IA

*L'argument classique* (Stanford, 1a inst., Laudan 1981) :

> *Nos meilleures théories scientifiques passées se sont toutes avérées fausses. Donc nos théories actuelles le seront probablement aussi.*

*Version appliquée à l'IA* :

> *L'IA n'a jamais pu accomplir des tâches de complexité X. Donc elle ne le pourra probablement jamais.*

*Asymétrie* : chaque franchissement de seuil est **irréversible**.

- On ne peut pas "désapprendre" que le #205 ($\Pi_3^0$) a été résolu par un LLM.
- On ne peut pas ignorer que Claude a résolu un problème ouvert de TAOCP en 1h.
- La méta-induction pessimiste appliquée à l'IA se *réfute elle-même* à chaque nouveau seuil franchi.

# Vers une hybridation ?

*Bilan :*

- Résolution autonome d'énoncés hautement complexes ($\Pi_3^0$ pour le #205).
- Collaboration itérative humain-machine (*Claude's Cycles* : 31 explorations).
- Continuité entre sémantique latente (compilateur de pseudocode) et preuve formelle (Lean).

*Quelles perspectives pour le chercheur ?*

- **Le rôle humain** : formulation de problèmes, architecture, jugement esthétique.
- **Le LLM comme *exosomatisation* mathématique** (Stiegler) : délégation partielle de la charge combinatoire.
- **Explication** : comment expliquer une preuve trouvée et certifiée de manière purement computationnelle ?

# Références


- Couturat, L. (1901). *La Logique de Leibniz d'après des documents inédits*. Paris : Alcan.
- van Heijenoort, J. (1967). "Logic as Calculus and Logic as Language." *Synthese* 17, 324–330.
- Knuth, D. E. (2026). *Claude's Cycles*. Stanford CS.
- Dean, W. & Naibo, A. (2025). *Artificial Intelligence and Inherent Mathematical Difficulty*. Philosophia Mathematica.
- Henkel, J. (2025). *The Mathematician's Assistant*. Math Semesterber 72, 117–144.
- Pommeret, L. (2026). Dépôt *pseudo-code* : `https://gitlab.lisn.upsaclay.fr/pommeret/pseudo-code`

# Les questions sans réponse de la RMS (été 2026)

# Le terrain : la rubrique QR de la RMS

**QR**

rubrique *Questions-Réponses* de la *RMS*

**CPGE**

des questions posées par des enseignants de prépa

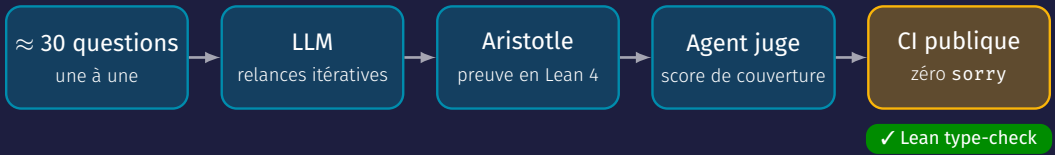
**≈ 100**

questions restées sans réponse

Aucun enjeu professionnel : le plaisir de la réflexion.

Non des controverses à clore, mais des **questions à habiter**.

# L'expérience




**Temps**

une nuit

**Coût**

< 20 €

# La grille (1/3) : à nu

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| 565 | 587 | 604 | 655 | 668 | 701 | 706 | 728 | 730 | 748 | 756 |
| 759 | 764 | 766 | 776 | 781 | 788 | 803 | 804 | 805 | 830 | 831 |
| 838 | 839 | 850 | 855 | 857 | 865 | 867 | 877 | 885 | 896 | 899 |

production · validation · circulation

# La grille (2/3) : production

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| 565 | 587 | 604 | 655 | 668 | 701 | 706 | 728 | 730 | 748 | 756 |
| 759 | 764 | 766 | 776 | 781 | 788 | 803 | 804 | 805 | 830 | 831 |
| 838 | 839 | 850 | 855 | 857 | 865 | 867 | 877 | 885 | 896 | 899 |

A : solution complète  C : progrès substantiel

## 33 traitées

dont 29 avec
couverture Lean complète

## échecs gradués

couverture partielle
Q706, Q748

## redécouvertes

Q730 : Weyl–Horn
Q804 : Tammes

# Pivot : Dean & Naibo (2025)

**Thèse**

La difficulté mathématique est **inhérente**, pas un obstacle contingent.

**Mon corpus ne la teste pas**

Sous le niveau de la recherche, sans doute dans les données d'entraînement.

**Ce que le cas révèle**

La difficulté non comme obstacle, mais comme **bien**, choisi par une communauté.

# La contrainte selon Dean & Naibo : la forme logique

**$\Sigma_1^0$** $\exists x\, \varphi(x)$

Il existe un objet : il suffit de l'**exhiber**.

**$\Pi_1^0$** $\forall x\, \varphi(x)$

Pour tout $n$, cela tient. Réfuter : **un contre-exemple**.

**$\Pi_2^0$** $\forall x\, \exists y\, \varphi$

Quantificateurs alternés : il faut un **argument général**.

Thèse : l'IA excelle dans la **recherche de témoin** ($\Sigma_1^0$), pas au-delà.

# Ce que Lean garantit

**✓ Garanti**

La **correction formelle**, qu'aucune relecture humaine ne garantit.

**✗ Non garanti**

Que l'énoncé formalisé soit **celui posé en français**.

La fidélité reste une affaire humaine.

C'est là que se logent les échecs les plus instructifs.

# Du français à Lean : Q728, le jeu du bâton

## Énoncé imprimé (A. Tissier, RMS 121-4)

Un bâton de $n$ cm ; A et B le cassent tour à tour en morceaux entiers non nuls *distincts*. À la fin, $x$ bouts de 1 cm, $y$ bouts de 2 cm. Vrai ou faux : pour tout $n$, A gagne si $n \equiv 2$, B gagne si $n \equiv 1$, nulle si $n \equiv 0 \pmod 3$ ?

| FRANÇAIS | LEAN 4 |
|---|---|
| un bâton, puis des morceaux | `P : Multiset ℕ`, départ `{n}` |
| casser $m$ en deux entiers non nuls distincts | `0 < a ∧ a < b ∧ a + b = m` |
| $x$ bouts de 1 cm, $y$ bouts de 2 cm | `P.count 1`, `P.count 2` |
| « on suppose qu'ils jouent au mieux » | `def Good` (récurrence bien fondée) |
| « pour tout $n$ » | `(hn : 3 ≤ n)` $n = 1, 2$ : aucun coup jouable |

# Faut-il envoyer ?

**Julien Bernis**

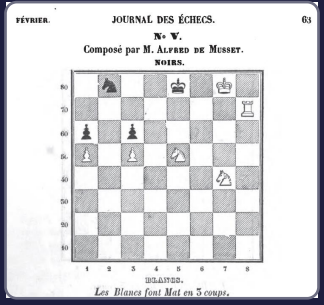
Favorable, mais : comment valoriser ?
Quel accueil chez les pairs ?

**Sa suggestion**

Montrer non ce qui est résolu, mais **ce qui résiste**.

Son analogie : résoudre le problème d'échecs du journal avec Stockfish.

# Une confusion d'époques


FÉVRIER. JOURNAL DES ÉCHECS. 63

N° V.

Composé par M. ALFRED DE MUSSET.

NOIRS.

BLANCS.

*Les Blancs font Mat en 5 coups.*

Alfred de Musset, *Journal des échecs*

avant Stockfish un problème **public**, posé dans un journal — comme la RMS

après un problème **individuel** : se forcer à ne pas voir la solution

# La grille (3/3) : ce qui résiste

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| 565 | 587 | 604 | 655 | 668 | 701 | 706 | 728 | 730 | 748 | 756 |
| 759 | 764 | 766 | 776 | 781 | 788 | 803 | 804 | 805 | 830 | 831 |
| 838 | 839 | 850 | 855 | 857 | 865 | 867 | 877 | 885 | 896 | 899 |

partie encore ouverte résolue

Ce qui reste : pour qui ? pour quoi ?

On ne peut pas désapprendre
que ces questions tombent
en une nuit pour vingt euros.

Quoi que décide la rédaction,
la rubrique ne peut plus être ce qu'elle était avant.

# Question à la salle

Que devient une pratique intellectuelle gratuite
lorsqu'un outil la rend soluble à la demande,
pour le prix d'un abonnement ?

# Bouleversements récents



8 septembre 2026 Recherches Publication

## À propos du problème du prix du millénaire de Navier–Stokes

Lire l'article Accéder à la démonstration formalisée dans Lean >

Écouter l'article 11:00 Partager

Nous présentons une solution au problème d'existence et de régularité de Navier–Stokes, l'un des problèmes du prix du millénaire. Cette démonstration, produite par un système interne d'OpenAI, montre que la dynamique des équations de Navier–Stokes décrivant le

# Bouleversements récents



vices

Le Monde

FR | EN

Menu

Mobilisations sociales Ukraine International Planète Politique Société Économie Idées Culture Le Goût du Monde Sciences Sport Pi

SCIENCES • MATHÉMATIQUES

## Nouveau coup de massue d'OpenAI sur les mathématiciens : « Ils ont annihilé mon domaine de recherche »

**Après les dernières annonces de l'entreprise américaine sur la résolution de plusieurs centaines de problèmes de maths, le médaillé Fields Hugo Duminil-Copin évoque un « démantèlement » de sa discipline. Il craint en particulier les conséquences de l'intelligence artificielle sur l'avenir des jeunes.**

Par David Larousserie

Publié le 10 octobre 2026 à 06h30, modifié le 12 octobre 2026 à 17h45 · Lecture 3 min. · Read in English

Article réservé aux abonnés

Lire plus tard

Mettre *Le Monde* en favori sur Google

# Pause

Reprise dans 10 minutes

Ensuite : LLM agentique et vibe coding (25 min), puis discussion (40 min)

# LLM agentique

# Appel d'outil (*function calling*)

- LLM seul : du texte, rien d'autre (il ne lit rien, n'exécute rien)
- Outils décrits dans la requête : nom, description, schéma JSON des arguments
- Réponse : un appel d'outil au lieu d'une phrase → l'hôte exécute l'appel et renvoie le résultat au LLM

Toolformer (2023), puis le *function calling* (juin 2023), devenu le format des API « compatibles OpenAI » (dont Aristote)

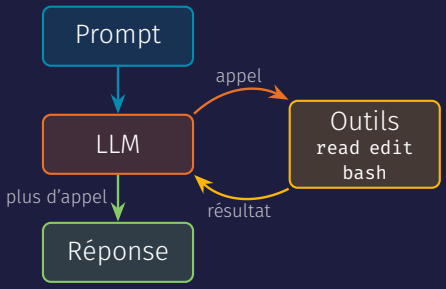
```
// 1. la requête décrit l'outil
"tools": [{"type": "function", "function": {
  "name": "lemmatiser",
  "description": "Lemmes d'une forme grecque",
  "parameters": {"type": "object", "properties":
    {"forme": {"type": "string"}}}}}]

// 2. le modèle répond par un appel, sans texte
"tool_calls": [{"id": "call_1",
  "type": "function", "function": {
    "name": "lemmatiser",
    "arguments": "{\"forme\": \"ἀνθρώπων\"}"}}]

// 3. l'hôte exécute, puis renvoie le résultat
{"role": "tool", "tool_call_id": "call_1",
 "content": "ἄνθρωπος"}
```

ἀνθρώπων : génitif pluriel de ἄνθρωπος (« homme »)

# Boucle d'agent




```python
messages = [systeme, consigne]
while True:
    rep = llm(messages, OUTILS)  # un tour
    messages.append(rep)
    if not rep.tool_calls:       # plus d'appel :
        return rep.content       # c'est fini
    for appel in rep.tool_calls:   # l'hôte agit
        r = OUTILS[appel.name](**appel.args)
        messages.append(resultat(appel.id, r))
```

*Workflow* : le code fixe le chemin

*Agent* : le LLM choisit ses outils, et quand s'arrêter

« An LLM agent runs tools in a loop to achieve a goal. » (S. Willison, 2025)

# Contexte




prompt système et outils, renvoyés à chaque tour

un fichier lu, un journal, une page web : ce qui ne sert plus coûte quand même

262 144 tokens (Qwen, Gemma)
131 072 (Llama 3.1 8B)

## Limites

- Performances en baisse quand le contexte s'allonge, même sur des tâches simples (et le milieu d'un long contexte est moins bien utilisé)
- 11 outils MCP déclarés → 1 600 à 2 200 tokens, à chaque requête

## Remèdes

- Une tâche par conversation (`/clear`)
- L'état du travail dans un fichier (`PLAN.md`)
- Des sous-agents qui ne renvoient qu'un compte rendu

Hong, Troynikov et Huber (Chroma, 2025) ; Liu et al. (TACL, 2024) ; Rajasekaran et al. (Anthropic, 2025)

# pi et pi-aristote

## pi (noyau minimal)

- Quatre outils par défaut (`read`, `write`, `edit`, `bash`) ; lit le fichier `AGENTS.md` du projet
- Aucune demande de permission → dossier dédié, git
- Le reste en extensions TypeScript : fournisseurs, commandes, MCP

## Extension pour Aristote

- μ Liste et sonde les modèles, masque ceux en panne
- ρ Relance les réponses vides (4 fois au plus)
- κ Ajuste `max_tokens` à la place restante
- Φ Relance un LLM qui *annonce* une action sans appeler d'outil

Les quatre corrections de l'extension portent sur la boucle, aucune sur le LLM

pi 1.0.0 : Mario Zechner, Earendil, licence MIT · pi-aristote : code ouvert sur GitHub

# Vibe coding

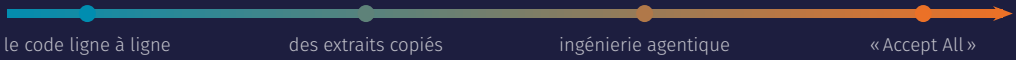
# Vibe coding (Karpathy, février 2025)




- **2 février 2025**
  le message (plus de 7 millions de vues)
- **6 novembre 2025**
  « vibe coding », mot de l'année du Collins
- **4 février 2026**
  un autre nom, pour l'usage professionnel :

agentic engineering

# Vibe coding vs ingénierie




vers la droite, on délègue davantage et on relit de moins en moins

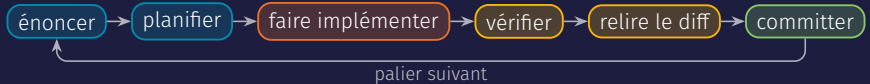
- Vibe coding : déléguer sans relire, et juger au résultat visible
- Relu, testé, explicable : « ce n'est pas du vibe coding, c'est du développement logiciel » (Willison, 2025, trad.)

- *Bien pour* : prototypes, scripts jetables, figures
- *Risqué pour* : le code dont sortent des résultats publiés, des données qu'on ne peut pas recréer

**METR (2025)**

16 développeurs expérimentés, 246 tâches dans leurs propres dépôts : avec IA, 19 % plus lents, alors qu'ils pensaient avoir gagné 20 %

# Bonnes pratiques




```
## Ce que tu ne dois pas faire
- Modifier ou affaiblir des tests existants
  pour les faire passer.
- Inventer une référence bibliographique.
- `git push`, `git reset --hard`, `rm -rf` :
  jamais sans accord explicite.
```

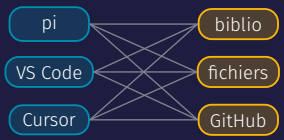
Extrait abrégé du gabarit `AGENTS.md` du wiki « Vibecoding au LISN »

- Énoncer le résultat observable et borner le périmètre
- `AGENTS.md` (comment lancer, structure, interdits), que pi lit seul ; `PLAN.md` pour le but, les étapes à cocher et les notes
- git : petits commits, `git diff` avant chacun
- Vérifier soi-même : « lance-le et montre-moi la sortie »

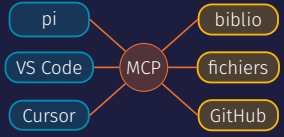
# MCP (Model Context Protocol)

# MCP (Anthropic, novembre 2024)

sans protocole : $N \times M$



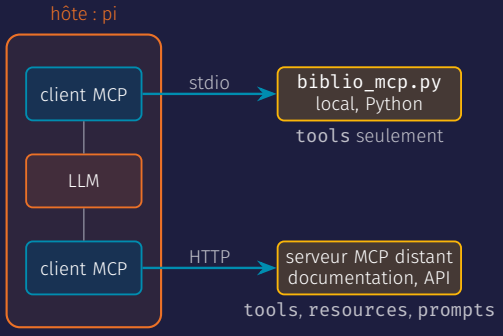

avec MCP : $N + M$




- 25 novembre 2024 : publié par Anthropic, en open source
- Mars 2025 : adopté par OpenAI ; aujourd'hui aussi par Cursor, VS Code, Gemini, Copilot…
- 9 décembre 2025 : confié à l'Agentic AI Foundation (Linux Foundation), avec `AGENTS.md`
- 2026 : spécification `2026-07-28` ; intégré à pi (version 0.99, septembre)

Sur le modèle du Language Server Protocol des éditeurs (la documentation parle d'« un port USB-C pour les applications d'IA »)

# Hôte, clients, serveurs MCP




- Messages en JSON-RPC 2.0, un client par serveur MCP
- Trois primitives, selon qui décide :

| | | |
|---|---|---|
| `tools` | fonctions | le LLM |
| `resources` | données (URI) | l'application |
| `prompts` | gabarits | l'utilisateur |

- *stdio* : un sous-processus, une ligne JSON par message
- *Streamable HTTP* : un POST par message, OAuth

L'hôte garde la conversation (un serveur MCP ne la lit pas et ne voit pas les autres serveurs)

# Serveur MCP en Python (`biblio_mcp.py`)

```python
from fastmcp import FastMCP
mcp = FastMCP("biblio-mcp")

@mcp.tool()
async def format_bibtex(identifier: str,
        source: str = "auto") -> dict:
    """Fetch an article and format as BibTeX.

    Args:
        identifier: DOI, arXiv ID, or URL
                    of the article.
    """
    ...

mcp.run()       # transport stdio par défaut
```

Ce que le LLM reçoit (réponse à `tools/list`)

```
{"name": "format_bibtex",
 "description": "Fetch an article and
                 format as BibTeX.",
 "inputSchema": {"type": "object",
   "properties": {"identifier": {
     "type": "string", "description":
     "DOI, arXiv ID, or URL of the
     article."},
     ...},
   "required": ["identifier"]}}
```

- *Docstring* et types → description et schéma lus par le LLM
- 11 outils, ni `resources` ni `prompts`

# Échange JSON-RPC et configuration de pi

```
>>> {"jsonrpc":"2.0","id":1,"method":"initialize","params":{...}}
<<< {"jsonrpc":"2.0","id":1,"result":{"protocolVersion":"2025-11-25",...}}
>>> {"jsonrpc":"2.0","id":3,"method":"tools/call","params":{"name":"format_bibtex",
      "arguments":{"identifier":"10.1162/tacl_a_00638"}}}
<<< {"jsonrpc":"2.0","id":3,"result":{"content":[{"text":"{\"status\":\"ok\",\"bibtex\":
      \"@article{liu2024lost,\\n  author = {Nelson F. Liu and ...},\\n  title = {Lost in the
      Middle: ...},\\n ... year = {2024},\\n  doi = {10.1162/tacl_a_00638},\\n ...}\"}",...
```

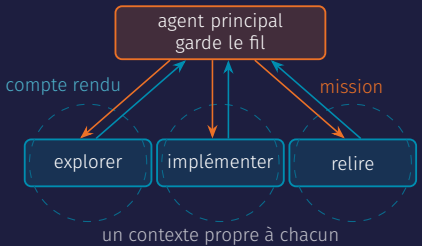
```
// ~/.pi/agent/mcp.json
{"mcpServers": {"biblio": {
  "command": "uv",
  "args": ["run", "--directory",
           "~/mcp-ecautal", "biblio-mcp"],
  "exposure": "direct"}}}
```

```
$ pi mcp list
biblio: connected, 11 tools (direct, global)
```

- Dans pi : l'outil `mcp__biblio__format_bibtex`
- Première version : DOI vide partout (clé `DOI` lue en minuscules, un vrai bug) → un serveur MCP se teste comme le reste
- Un serveur MCP, c'est du code qui tourne sur la machine → prudence avec les serveurs d'autrui

# Sous-agents

# Sous-agents (Anthropic et Cognition, juin 2025)




- *Sous-agent* : une autre boucle, avec son propre contexte, ses outils et son prompt système, qui ne renvoie qu'un compte rendu
- *Intérêt* : isoler le contexte, paralléliser, vérifier (un relecteur qui n'a pas écrit le code), restreindre les droits
- *Coût* : environ 4 fois les tokens d'un chat pour un agent, 15 fois pour plusieurs agents
- *Échecs* : des décisions implicites incompatibles entre agents (14 modes d'échec recensés)

Oui pour des tâches séparables, non pour des décisions couplées

Hadfield et al. (Anthropic, 2025) ; Yan (Cognition, 2025) ; Cemri et al. (NeurIPS, 2025)

# Ultracode (le Lycée d'Aristote)

| | Disciple | Mission | Outils (en plus de `read grep find ls`) |
|---|---|---|---|
| π | Péripatéticien | explore, sans rien modifier | `bash` |
| ε | Euclide | trace le plan d'implémentation | aucun |
| η | Héphaïstos | implémente, sur ses seuls fichiers | `bash edit write` |
| σ | Socrate | relit et lance les tests, sans corriger | `bash` |
| λ | Sappho | écrit la documentation | `bash edit write` |

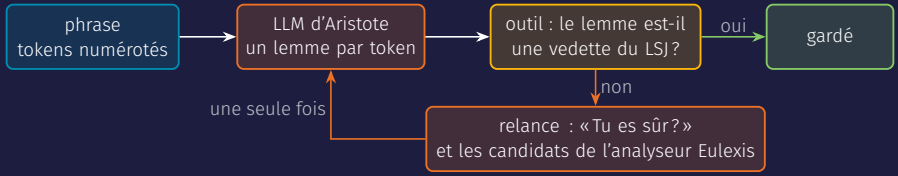
- `Alt+U`, `/ultracode` ou le mot « ultracode » dans un prompt → le LLM reçoit l'outil `agents`
- Chaque mission → un sous-agent (processus pi enfant) qui ne voit pas la conversation : tout lui redire dans la mission
- 8 missions au plus, 4 en parallèle ; l'agent principal synthétise les comptes rendus

## Limites

- Tous dans le même dossier
- Pas de délai maximal (`Échap` pour arrêter)
- Lecture seule garantie pour Euclide seulement

# Lemmatiser le grec ancien

# Banc de lemmatisation (septembre 2026)



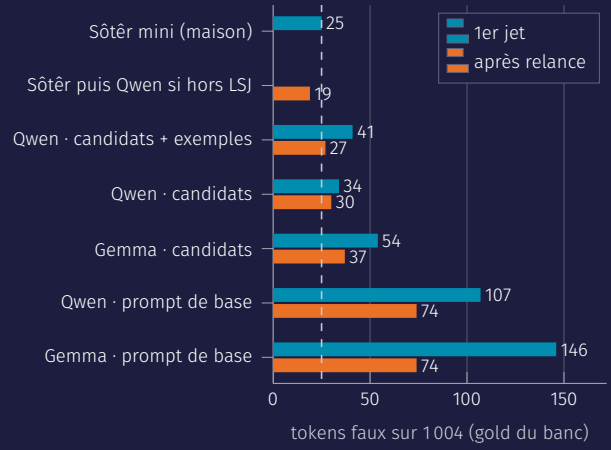

## Tâche

- De la forme au lemme du LSJ : καθεῖλεν → καθαιρέω (« abattre »)
- 58 phrases, d'Homère à Plutarque (1 004 tokens notés)
- À battre : Sôtêr mini, modèle maison et local, 97,5 % en 18 s

## Relance « tu es sûr ? »

- Qwen 3.6 35B, 61 tokens relancés : 32 corrigés, 0 cassé
- Qwen : 89,3 % → 92,6 % ; Gemma 4 31B : 85,5 % → 92,6 %
- Par exemple καθεΐζω (n'existe pas) → καθαιρέω

# Résultats du banc




- *Contexte* : les candidats d'Eulexis dans le prompt → Qwen passe de 107 à 34 erreurs
- *Relance* : utile surtout sans candidats (Gemma : 146 → 74)
- *Hybride* (Sôtêr, puis Qwen quand Sôtêr sort du LSJ) : 98,1 %

Pointillé : Sôtêr mini (25 erreurs) ; un point = 10 tokens, et sous un point l'écart n'est pas significatif

# Gold du banc (fabriqué et attaqué par des agents)

## Fabriquer

- 12 agents LLM en parallèle (6 lots, 2 annotateurs indépendants par lot, un protocole écrit)
- La boucle du banc : un script signale les lemmes hors LSJ, l'agent corrige ou justifie
- Accord entre annotateurs : 99,6 %

## Attaquer

- 3 agents « hostiles » en lecture seule sur la liste de fréquence du site, sans leur dire quoi chercher
- Trouvé : εἶπον au rang 48, un aoriste (de λέγω, « dire ») compté comme lemme

τὸ μὲν οὔ ποτε φύλλα καὶ ὄζους / φύσει (*Iliade*, I, 234–235)

« ne produira plus feuilles ni rameaux »

φύσει : futur de φύω (« produire »), pas le datif de φύσις (« nature »)
→ vu par les deux agents annotateurs, raté par Sôtêr

# Discussion

# Bilan

# Bilan

- Un agent, c'est un LLM, des outils et une boucle :
  pi-aristote, installé à l'accueil, en est un
- Dans le contexte, la bonne information, pas toute (avec les candidats d'Eulexis dans le prompt, Qwen passe de 107 à 34 erreurs)
- Un serveur MCP tient en une fonction Python décorée, et se teste comme le reste (le DOI vide de la première version de `biblio_mcp.py`)
- Ce que l'agent écrit se vérifie par un outil : les tests et `git diff` à chaque palier du TP, les vedettes du LSJ dans mon banc (32 lemmes corrigés, 0 cassé)

# Liens et lectures

## Liens

- pi-aristote : `vibe-coding.pages.lisn.upsaclay.fr/pi-aristote`
- Code : `gitlab.lisn.upsaclay.fr/vibe-coding/pi-aristote`
- pi : `pi.dev` ; MCP : `modelcontextprotocol.io`
- `AGENTS.md` : `agents.md`

## Lectures


- Schluntz et Zhang, *Building effective agents* (Anthropic, 2024)
- Willison, *Not all AI-assisted programming is vibe coding* (2025)
- Yan, *Don't Build Multi-Agents* (Cognition, 2025)


Références complètes en annexe C

# Merci pour votre `attention` !

Avez-vous des questions ?



`vibe-coding.pages.lisn.upsaclay.fr/pi-aristote`

# Annexes : sommaire

A. Commandes de pi-aristote 1.8.0 et raccourcis de pi
B. Brancher un serveur MCP dans pi
C. Références (4 pages)

# A. Commandes de pi-aristote 1.8.0 et raccourcis de pi

| Extension Aristote | Effet |
|---|---|
| `/aristote` | état des modèles, de la clé |
| `/aristote refresh` | revérifier les modèles |
| `/aristote key sk-...` | enregistrer une nouvelle clé |
| `/maj` | mettre à jour l'extension |
| `/langue auto\|fr\|en\|...` | langue des réponses |
| `/gardefou on\|off` | Φ garde-fou : activer, couper |
| `/ultracode on\|off` | Ω mode Ultracode (ou `Alt+U`) |
| `/socrate <mission>` | un disciple seul ; de même `/peripateticien`, `/euclide`, `/hephaistos`, `/sappho` |
| `/clear`, `/exit` | repartir de zéro (`/new`), quitter (`/quit`) |

| pi | Effet |
|---|---|
| `Échap` | interrompre l'agent |
| `Ctrl+L` | choisir le modèle (`/model`) |
| `Ctrl+P` | modèle suivant |
| `Maj+Tab` | niveau de réflexion |
| `Ctrl+O` | déplier les sorties d'outils |
| `/compact` | résumer le contexte |
| `/mcp` | serveurs MCP de la session |
| `/reload` | tout recharger |

Pas de commande pour le navigateur (il suffit de demander « cherche sur le web… »)
Relances de l'extension visibles avec `ARISTOTE_DEBUG=1 pi`

# B. Brancher un serveur MCP dans pi

Dans un terminal (hors de pi)

```
$ pi mcp add biblio --exposure direct \
    -- uv run --directory ~/mcp-ecautal \
    biblio-mcp
$ pi mcp list
biblio: connected, 11 tools (direct, global)
$ pi mcp remove biblio
```

- Fichiers : `~/.pi/agent/mcp.json` ; `.pi/mcp.json` dans un projet de confiance
- Dans pi : `/mcp` pour l'état, `/reload` après une modification
- Outils nommés `mcp__<serveur>__<outil>`

| Exposition | Ce que voit le LLM |
|---|---|
| `codemode` (défaut) | rien : il écrit un script qui appelle les outils |
| `deferred` | outils chargés par recherche |
| `direct` | déclarés comme outils |
| `hidden` | rien |

Avec les modèles d'Aristote, `direct` est le plus sûr pour un petit serveur MCP (vérifié le 23/10/2026 avec pi 1.0.0 et `biblio_mcp.py`, FastMCP 4.1.3)

# C. Références (1/4) : articles


Articles

- Yao, S. et al. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023. arXiv:2210.03629
- Schick, T. et al. (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools*. NeurIPS 2023. arXiv:2302.04761
- Yang, J. et al. (2024). *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. NeurIPS 2024. arXiv:2405.15793
- Liu, N. F. et al. (2024). *Lost in the Middle: How Language Models Use Long Contexts*. TACL, 12, 157–173. doi:10.1162/tacl_a_00638
- Becker, J., Rush, N., Barnes, E., Rein, D. (2025). *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity*. METR. arXiv:2507.09089
- Cemri, M. et al. (2025). *Why Do Multi-Agent LLM Systems Fail?* NeurIPS 2025, Datasets and Benchmarks. arXiv:2503.13657

Rapports

- Hong, K., Troynikov, A., Huber, J. (2025). *Context Rot: How Increasing Input Tokens Impacts LLM Performance*. Chroma. trychroma.com/research/context-rot
- Schluntz, E., Zhang, B. (2024). *Building effective agents*. Anthropic. anthropic.com/engineering/building-effective-agents

# C. Références (2/4) : rapports et billets

Rapports (suite)


- Hadfield, J. et al. (2025). *How we built our multi-agent research system*. Anthropic. anthropic.com/engineering/multi-agent-research-system
- Rajasekaran, P. et al. (2025). *Effective context engineering for AI agents*. Anthropic. anthropic.com/engineering/effective-context-engineering-for-ai-agents
- Jones, A., Kelly, C. (2025). *Code execution with MCP*. Anthropic. anthropic.com/engineering/code-execution-with-mcp


Billets


- Varda, K., Pai, S. (2025). *Code Mode: the better way to use MCP*. Cloudflare. blog.cloudflare.com/code-mode
- Yan, W. (2025). *Don't Build Multi-Agents*. Cognition. cognition.com/blog/dont-build-multi-agents
- Willison, S. (2025). *Not all AI-assisted programming is vibe coding (but vibe coding rocks)*. simonwillison.net/2025/Mar/19/vibe-coding
- Willison, S. (2025). *The lethal trifecta for AI agents*. simonwillison.net/2025/Jun/16/the-lethal-trifecta
- Willison, S. (2025). *I think "agent" may finally have a widely enough agreed upon definition to be useful jargon now*. simonwillison.net/2025/Sep/18/agents

# C. Références (3/4) : billets et annonces


Billets (suite)

- Zechner, M. (2025). *What if you don't need MCP at all?* `mariozechner.at/posts/2025-11-02-what-if-you-dont-need-mcp`
- Zechner, M. (2025). *What I learned building an opinionated and minimal coding agent.* `mariozechner.at/posts/2025-11-30-pi-coding-agent`

Messages, annonces, dictionnaires

- Karpathy, A. (2 février 2025). Message sur X définissant le « vibe coding ». `x.com/karpathy/status/1886192184808149383`
- Karpathy, A. (4 février 2026). Rétrospective, « agentic engineering ». `x.com/karpathy/status/2019137879310836075`
- Collins Dictionary (6 novembre 2025). *Collins' Word of the Year 2025.*
- Anthropic (25 novembre 2024). *Introducing the Model Context Protocol.* `anthropic.com/news/model-context-protocol`
- Soria Parra, D. (9 décembre 2025). *MCP joins the Agentic AI Foundation.* `blog.modelcontextprotocol.io`
- OpenAI (13 juin 2023). *Function calling and other API updates.* (date confirmée par une source secondaire)

# C. Références (4/4) : documentation et travaux cités

Spécifications et documentation

- *Model Context Protocol Specification*, révision 2026-07-28 : `modelcontextprotocol.io/specification/2026-07-28`
- `AGENTS.md` : `agents.md`
- pi 1.0.0 (`@earendil-works/pi-coding-agent`, licence MIT) : `pi.dev` et le dossier `docs/` du paquet
- Aristote (CentraleSupélec) : `aristote.education/llm-access/`

Mes travaux cités

- Banc de lemmatisation du grec ancien par LLM (16 septembre 2026) : rapport et gold
- Serveur MCP `biblio_mcp.py` (FastMCP) ; wiki « Vibecoding au LISN » (gabarits `AGENTS.md`, `PLAN.md`)
- pi-aristote : `gitlab.lisn.upsaclay.fr/vibe-coding/pi-aristote`